



TED UNIVERSITY
Faculty of Engineering

Department of Computer Engineering

CMPE 492 Senior Project

ARIS

Test Plan

Spring 2026

Taha Akdemir | ID: 12577008730

Ecem Nur Bilgi | ID: 14830754170

Şevkiye Sıla Kahya | ID: 14620089482

Elif Zeynep Elverişli | ID: 13942218426

Table of Contents

1. Introduction	2
1.1 Scope	2
1.2 Objectives	3
2. Features to be Tested	4
2.1 Input Acquisition and Buffering	4
2.2 Recognition Component	4
2.3 Command Processing	5
2.4 Communication System	5
2.5 Security and Access Control	6
2.6 Web Interface and Monitoring	7
2.7 Web-Based Control and Remote Operation	7
2.8 Error Handling and Recovery	8
3. Testing Methodology	8
3.1. Unit Testing:	8
3.2. Integration Testing:	9
3.3. System Testing:	10
3.4. Performance Testing:	11
3.5. User Acceptance Testing (UAT):	12
3.6. Beta Testing:	12
4. Test Environment	13
5. Sample Test Case	15
6. Conclusion	18
7. References	19

1. Introduction

This test plan explains how the ARIS (AI-Assisted Smart Helmet Communication System) will be tested. The main goal of this document is to make sure that the system works correctly and can operate properly in real-time conditions.

ARIS is a smart helmet system that allows hands-free communication by interpreting user inputs such as gestures, facial expressions and voice commands. The system is built on a Raspberry Pi-based helmet unit and includes components like input acquisition, recognition, command processing, communication and a web-based monitoring and control interface.

1.1 Scope

The test plan scope will examine all major elements and functionality of the ARIS system. The testing process will evaluate both functional and non-functional elements to determine system performance in independent operation and complete system integration.

The scope includes:

- **Input Acquisition and Buffering**
This section describes how visual and audio data collection occurs through helmet-based systems which use their camera and microphone. System should be able to capture data continuously, synchronize inputs and store them temporarily without losing information.
- **Recognition of User Inputs**
The system must correctly identify three types of input which include handheld gestures and facial expressions and vocal commands. The system needs to maintain accurate performance results while it processes various user behavior patterns.
- **Command Processing and Validation**
The system requires input recognition to transform recognized data into operational commands. The system needs to block the execution of both invalid and incomplete input.
- **Communication Between System Components**
The helmet unit needs to maintain stable communication with both the backend system and the web dashboard. The system needs to keep functioning through minor network interruptions.
- **Data Transmission and Visualization**
The web dashboard needs to receive visual data which should display properly on the platform. The system requires updates that approximate real-time performance to enable efficient system monitoring.
- **Web-Based Monitoring and Control**
Users should be able to monitor the system and send commands through web control panel.

The interface should display the present system status and provide simple ways for users to interact with it.

- **Error Handling and System Stability**
The system needs to identify errors because it must display relevant error messages and it needs to handle issues which include losing network connections. The system needs to maintain its operations without experiencing system failures.

1.2 Objectives

The test plan aims to verify that the ARIS system meets its requirements while delivering reliable performance for real-time operations. The testing process evaluates both individual performance of each component and the system's operational performance when all components function together.

The objectives are:

- **Verification of System Components:** It is expected that all such components may perform remarkably and integrate perfectly with one another.
- **Accurate Interpretation of User Inputs:** System should also correctly understand gestures, facial expressions and voice commands.
- **Correct Command Generation and Execution:** Commands should be generated correctly and executed without causing unexpected behavior.
- **Evaluation of Communication and Data Transmission:** System should transfer data and commands properly between components without errors.
- **Real-Time Operation with Acceptable Latency:** System should respond quickly and not cause noticeable delays during use.
- **System Stability Under Different Conditions:** System should remain stable even under different conditions like high usage or connection issues.
- **Identification of Issues and Improvement of Reliability:** Testing should help find problems and improve the overall reliability of the system.

2. Features to be Tested

This section explains the main features of the ARIS system that will be tested. These features are selected based on the system components and are used to check whether the system works properly, stays reliable and performs well in real time use.

2.1 Input Acquisition and Buffering

Description:

This component collects user input through the Raspberry Pi-based helmet unit and prepares it for further processing. It captures visual data using the camera and audio data using the microphone, and then organizes this data before passing it to the next stage.

Features to Test:

- **Camera Initialization and Capture (Test ID: INP-001):** The camera should start properly and continuously capture frames, ensuring that visual data is always available.
- **Microphone Input Capture (Test ID: INP-002):** The system should capture audio input clearly and in real-time.
- **Input Buffering (Test ID: INP-003):** Incoming data should be temporarily stored without any loss.
- **Partial Input Handling (Test ID: INP-004):** The system should continue operating even if one input type is unavailable.

Rationale:

Since all system operations depend on input data this component needs to work consistently.

2.2 Recognition Component

Description:

This component processes multimodal input using AI-based models to understand user's intention.

Features to Test:

- **Gesture Recognition (Test ID: REC-001):** The system should correctly detect predefined gestures.
- **Facial Expression Recognition (Test ID: REC-002):** Facial expressions should be identified accurately.

- **Voice Command Recognition (Test ID: REC-003):** Spoken commands should be interpreted correctly.
- **Recognition Accuracy (Test ID: REC-004):** Results should be consistent and reasonably accurate.
- **Low Confidence Handling (Test ID: REC-005):** The system should handle uncertain results without producing incorrect outputs.

Rationale:

Since user interaction depends on recognition this component must perform accurately.

2.3 Command Processing

Description:

This component converts recognized inputs into system commands using validation and mapping steps.

Features to Test:

- **Command Validation (Test ID: CMD-001):** Invalid or incomplete inputs should be rejected before processing.
- **Command Mapping (Test ID: CMD-002):** Valid inputs should be mapped to the correct commands.
- **Command Consistency (Test ID: CMD-003):** The same input should always produce the same output.
- **Error Handling in Processing (Test ID: CMD-004):** The system should handle unclear or conflicting inputs properly.

Rationale:

Incorrect commands directly affect system behavior and may cause unexpected behavior, so this step is important.

2.4 Communication System

Description:

This component manages communication between the helmet unit, backend and web dashboard including command transfer and data transmission to web interface.

Features to Test:

- **Command Transmission (Test ID: COM-001):** Commands should be sent to backend correctly.
- **Connection Handling (Test ID: COM-002):** Communication should remain stable during operation.
- **Message Buffering (Test ID: COM-003):** Data should be stored and resent after connection loss.
- **Session Management (Test ID: COM-004):** Sessions should be tracked and maintained correctly.
- **Communication Latency (Test ID: COM-005):** Response time should stay within acceptable limits.
- **Image/Data Transmission to Web Panel (Test ID: COM-006):** Visual data should reach the web interface with minimal delay.

Rationale:

Reliable communication is necessary for real time system behavior.

2.5 Security and Access Control

Description:

This component makes sure system security by controlling access and protecting data.

Features to Test:

- **Authentication Mechanism (Test ID: SEC-001):** Only authorized users and devices should be able to access the system.
- **Access Control (Test ID: SEC-002):** Users should only access features allowed by their roles.
- **Data Encryption (Test ID: SEC-003):** All communications must be encrypted safely.
- **Unauthorized Access Prevention (Test ID: SEC-004):** Any unauthorized access request must be prevented.

Rationale:

Security is important as the system involves real time data and control operations.

2.6 Web Interface and Monitoring

Description:

The web interface allows users to monitor the system through a web dashboard and view incoming data.

Features to Test:

- **Dashboard Display (Test ID: UI-001):** System status and data should be displayed clearly.
- **Log Retrieval (Test ID: UI-002):** Logs should be accessible and accurate.
- **Real-Time Data Display (Test ID: UI-003):** Visual data should be update continuously.
- **User Interaction (Test ID: UI-004):** The interface should respond smoothly and without delays.

Rationale:

This is the main interface so it should be simple and easy to use.

2.7 Web-Based Control and Remote Operation

Description:

System allows remote control through web interface which operators can monitor incoming data and send commands back to the system.

Features to Test:

- **Live Image Streaming (Test ID: WEB-001):** Images or video streams should be displayed correctly on the web interface.
- **Real-Time Updates (Test ID: WEB-002):** Data should update continuously without noticeable delay.
- **Remote Command Execution (Test ID: WEB-003):** Commands sent from the web control panel should be transmitted back to the system and executed correctly.
- **Interaction with Visual Data (Test ID: WEB-004):** Users should be able to interact with the displayed data when needed.
- **System-Web Synchronization (Test ID: WEB-005):** The system and interface should remain consistent.

Rationale:

It needs to work reliably because remote control is one of the key features.

2.8 Error Handling and Recovery

Description:

This component helps to ensure that any error will be managed effectively and maintain stability in the system.

Features to Test:

- **Error Detection (Test ID: ERR-001):** Errors should be detected correctly.
- **Error Messages (Test ID: ERR-002):** Error messages should be clear and easy to understand.
- **Recovery Mechanisms (Test ID: ERR-003):** The system should recover from failures.
- **System Stability (Test ID: ERR-004):** The system should remain stable under unexpected conditions.

Rationale:

A reliable system should handle errors without affecting its overall operation.

3. Testing Methodology

3.1. Unit Testing:

Description:

Unit testing refers to the process of testing the units of the ARIS software in isolation as mentioned in the low level design document. In this context, unit can refer to an independent program component such as a function, class or module. It is validated without any dependency from any other unit. The unit test ensures that the individual unit processes inputs, performs the recognition operations and manages the communication properly.

Approach:

• **Scope:** Unit testing would involve core modules like CameraManager (image capture), MicrophoneManager (microphone data), InputBuffer, Gesture Recognition, Voice Recognition modules and Communication Handler module.

- **Tools:** Java based backend functionalities will be validated with JUnit. Python modules would be validated with unittest or pytest framework. Mocking will be done for dependencies like hardware or networking communication.
- **Test Cases:** The test cases would be created in terms of not only functionality but also extreme cases. These test cases can include proper gesture recognition from valid input frames, proper handling of null values or corrupt input values to deal with noise in the audio input and proper command formation.
- **Execution:** The unit testing would be automated and performed continuously throughout the development process through CI/CD pipelines like GitHub actions to catch bugs early in the development process.
- **Metrics:** 90% of code coverage is expected to be achieved in critical code blocks. A code coverage tool would be used to make sure that critical logic paths are covered.

Rationale:

Through unit testing we can make sure that each module works fine before integrating it into the project and avoids any complexity during debugging.

3.2. Integration Testing:

Description:

The integration test ensures proper integration and data transfer between various components of the ARIS system. Since the ARIS system comprises several sub modules such as input modules, recognition modules based on AI, communication modules and also web modules. It is crucial to verify their flawless interaction.

Approach:

- **Scope:** The integration testing will be based on some major data flow paths such as:
 - Camera and microphone modules data flowing to recognition modules
 - Recognition modules output commands that come from gesture and voice recognition
 - Command generation module's output flows to the communication layer
 - Communication module sends data to the web interface
- **Tools:** Testing the API endpoints will be done using Postman. Interaction testing with the web interface using Selenium and custom scripts will test the data transfer between modules.

- **Test Cases:** Positive and negative test cases will both be carried out. Positive test cases will involve proper gesture recognition followed by proper transmission of commands. The negative test cases to be carried out will include delays in communication, improper gesture recognition, invalid format of data and interruptions in the network.
- **Execution:** Test cases will be implemented in staging environment that resembles real environment as much as possible.
- **Metrics:** It is important that all major integration flows be tested without any critical defects in communication and data exchange between the modules.

Rationale:

Integration testing is done to ensure that the various modules work together properly and there is proper flow of data throughout the system.

3.3. System Testing:

Description:

Testing of the ARIS system in its entirety, including all functional components from input devices to the communication module and web interface that is covered in system testing. The goal is to validate that the entire system operates well and is ready for integration into a full fledged production environment.

Approach:

- **Scope:** The following scenarios will be tested under system testing:
 - Proper recognition and command generation after receiving action or voice command from a soldier,
 - Correct communication of these commands via communication modules,
 - Displaying the received commands on the web interface. In addition, the system should respond correctly to cases where gestures or speech were not properly identified and/or commands failed to transmit.
- **Tools:** For testing the web interface, we will employ Selenium. Customising simulators will generate inputs for the camera/microphone and recognition modules, manual testing will be performed as well.
- **Test Cases:** Functional test cases would ensure correct system output while non-functional test cases would validate the performance requirements including response time of less than 5 seconds. Negative test cases would be carried out to determine how the system reacts to incorrect input.

- **Execution:** Test execution will be done in a controlled environment mimicking real life usage scenarios. Extended testing will also be executed to validate system reliability. Including tests lasting up to 8 hours.

- **Metrics:** The system should exhibit success rate of 95% in identifying and processing the commands entered into the system without causing any critical failure that affects its operation.

Rationale:

Testing is crucial in ensuring that all the requirements of the system have been met successfully. It is important in critical applications like this one where reliability is key.

3.4. Performance Testing:

Description:

Performance testing analyzes the behavior of the ARIS system during various types of operation: regular operation, continuous usage and high load operations.

Approach:

- **Scope:** Testing will include:

- Regular operation with regular input frequency;
- Continuous operation up to 8 hours;
- High load operation with high input and command generation frequencies.

- **Tools:** JMeter will be utilized to conduct the test, whereas other tools will monitor CPU usage, memory utilization and also latency.

- **Test Cases:** Response time of gesture and voice recognition should not exceed 2 seconds, whereas command transfer latency should take not more than 5 second. The test also aims at evaluating the throughput of the system.

- **Execution:** Tests will be carried out under the same conditions as in production environment.

- **Metrics:** Performance should be stable and also acceptable even during extreme loads.

Rationale:

The testing of performance serves as an effective means of ensuring the efficient operation of ARIS under actual conditions and its effectiveness in performing its intended functions under actual mission conditions.

3.5. User Acceptance Testing (UAT):

Description:

In User Acceptance Testing, the system will be evaluated from the perspective of its end users with regard to the system's usability and functionality to establish whether the system has met the required specifications. This test is carried out to establish whether the system has been usable and efficient for its intended purpose.

Approach:

- **Scope:** Testing will focus on issues such as usability testing, correctness of recognition, feedback and also responsiveness.
- **Participants:** Participants who will act like soldiers and system operator will engage in the process of UAT.
- **Tools:** Feedback will be gathered through the use of questionnaires and feedback sheets.
- **Test Cases:** Tests that will be performed will include issuing of commands in various condition using gestures and vocal control and web based interactions.
- **Execution:** UAT will be carried out in controlled testing environment.
- **Metrics:** The outcome expected will be high satisfaction levels from the user's perspective and usability evaluation.

Rationale:

User Acceptance Testing (UAT) is important for ensuring that the system meets user requirements, which is crucial for its successful operation in real life scenarios.

3.6. Beta Testing:

Description:

In the course of this testing the ARIS system will be tested by limited number of users in real life scenarios in order to detect possible flaws and receive some feedback before deploying the whole system.

Approach:

- **Scope:** Limited number of test cases in real life scenarios with small number of user will be applied in order to assess usability of the system.
- **Tools:** Bug trackers, usage log and other analytical tools will be used in the course of testing.

- **Test Cases:** Test cases will cover different scenarios for using the software, including stress testing and repeated testing.
- **Execution:** Implementation of the system in a real life setting under close supervision.
- **Metrics:** Target is obtaining of a stable working system with minimal number of critical bugs.

Rationale:

Beta testing provides valuable insights into real world system behavior, helping to identify hidden issues and improve overall system quality before deployment.

4. Test Environment

This segment identifies the hardware elements and software applications and computer networks used to conduct tests on the ARIS system. Definition of the testing environment involves defining the conditions that have to be sustained during the testing process because the real conditions should be duplicated while conducting the test process since ARIS acts as a prototype system.

Hardware

The testing environment consists of two elements which include the helmet unit and the server side.

Helmet Unit:

- Raspberry Pi 4 Model B
- Camera module for visual input capture
- Microphone for audio input capture
- Portable power supply / battery support
- Helmet-mounted prototype structure.

Server / Monitoring Side

- A laptop or desktop computer for backend and dashboard testing.
- The system requires at least 8 GB RAM and standard processing capacity.
- The system needs a display device which enables users to see the web interface and track system performance.

Software

- The software environment includes the modules required for recognition command processing and communication monitoring.
- The helmet side operation requires Raspberry Pi OS or another Linux-based operating system which runs on the Raspberry Pi 4 helmet unit.

- The recognition modules consist of Python-based gesture recognition and facial expression recognition and voice command processing modules.
- The backend system handles server-side communication and session management and logging and monitoring services which process validated commands.
- The web interface provides a browser-based dashboard that enables system monitoring and log visualization and remote control operations.
- The frontend technologies of web dashboard interface components use HTML CSS and JavaScript.
- The system includes security components which provide authentication and role-based access control and encryption for secure system usage.
- Testing Tools: Manual functional testing, browser testing, latency observation and log inspection tools.

Network

The network environment is used to test communication between the helmet unit, backend and web interface.

- Local Wi-Fi connection between the helmet and server
- TCP/IP-based communication
- Secure communication channel for encrypted data transmission
- Stable network for normal tests
- Temporarily interrupted network scenarios for recovery and buffering tests

Environment Setup

Before the tests start, the following setup conditions must be satisfied:

- The camera and microphone must be connected and active.
- The Raspberry Pi-based helmet system must be powered on.
- Recognition modules must be loaded correctly.
- The backend must be running and ready to receive data.
- The web interface must be accessible to authorized users.
- Predefined gesture, facial expression and voice command data must be available in the system.
- Logging and monitoring functions must be enabled.

Test Conditions

The ARIS system will be tested under different but controlled conditions such as:

- normal lighting conditions
- low or moderate background noise
- stable connection conditions
- temporary connection loss scenarios
- repeated command input conditions
- partial input availability such as missing audio or missing visual data

Since ARIS is an academic prototype, the test environment does not include actual military communication infrastructure or classified communication systems. All tests are performed in a controlled prototype environment suitable for development and validation purposes. This is also consistent with the scope and constraints of the ARIS project.

5. Sample Test Case

This section presents a sample test case for the ARIS system. The purpose of this test case is to check whether the system can correctly receive user input, recognize it, process it into a valid command and send it to the backend and web interface without problems. This test case covers the main system flow and reflects one of the most important use scenarios of ARIS.

Test Case ID: SYS-001

Title: Validate Recognition and Command Transmission

Feature

Input Acquisition and Buffering, Recognition Component, Command Processing, Communication System, Web Interface and Monitoring

Tested:

Test

To verify that the ARIS system can capture a predefined gesture or voice command, recognize it correctly, convert it into a valid system command, transmit it to the backend and display the result on the web dashboard.

Objective:

Test

System Testing

Type:

Preconditions

- The helmet unit is powered on.
- Camera and microphone are connected and functioning.

- Recognition modules are active.
- The communication system is connected to the backend.
- The web dashboard is open and accessible by an authorized user.
- The predefined input to be tested exists in the command set.
- Network connection is available.

Test Steps:

1. Turn on the ARIS helmet system.
2. Check if device appears as connected on web site.
3. Perform a predefined voice command.
4. Let the system take the input.
5. Observe if the recognition component identifies the input right.
6. Check whether the recognized input is processed as wrong input.
7. Check if the command is transmitted to the backend.
8. Check if command is displayed correctly on the web interface.
9. Confirm that the event is recorded in the logs.
10. Repeat the same test under slightly noisy or none-visibility conditions.

Input Data:

- One predefined gesture or voice command
- Active helmet device ID
- Active communication session
- Valid system connection

Expected Results:

- The system captures the input successfully.
- The recognition component identifies the gesture or voice command correctly.
- The command processing component converts the recognized input into the correct command.
- The communication system sends the command without interruption.

- The web dashboard displays the command and related system status correctly.
- A correct log entry is created for the event.
- The response is completed within acceptable time limits.
- If the recognition confidence is low, the system should not send an incorrect command and should handle the situation safely.

Actual Results:

It was successful in connecting to the helmet unit, which could be seen on the web dashboard. Once the command had been uttered, it was detected and processed by the system. The command was verified and sent to the back end without any issue. The command was shown properly on the dashboard, and even registered on the logs.

Pass/Fail Criteria:

The Pass/Fail Criteria states that there are two specific situations where the outcome can be either system pass or fail according to its operations. This means that the system will perform effectively when it is able to process user inputs in a comprehensive manner with accurate command generations and transmission from the back-end to display on the web interface.

On the other hand, the system is considered to be in failure state when either of the following happens – unable to receive user inputs correctly or unable to interpret them into commands leading to failure to execute and show them on the dashboard as well as compromising on system stability.

Test Environment:

The testing environment uses Raspberry Pi 4 with camera and microphone equipment together with server-side computer system and recognition and communication and backend and web dashboard software through a protected local Wi-Fi network that connects both helmet users and approved monitoring users.

Roles and Responsibilities:

The test execution is performed by a team member who uses the helmet while the test observer monitors dashboard and system response for observation purposes and the test recorder documents all results together with timing information and potential errors.

Risks:

The first risk states that background noise will interfere with the ability to recognize human speech. The second risk states that poor lighting conditions will make it difficult to detect gestures and facial features. The third risk states that the system will experience command transmission delays

due to temporary connection interruptions. The system will reject low-confidence input which users provide through their input.

The test case is critical because it verifies the primary functioning process of ARIS system operations. The core system behavior testing reveals whether the project which depends on user input recognition for sending commands operates correctly and reliably.

6. Conclusion

The test plan establishes a systematic testing method which assesses the ARIS system and its essential components through testing in controlled environments. The testing process aims to confirm that the system performs its functions accurately and with dependable operation during actual operating conditions.

The plan defines all essential components of the ARIS system which include input acquisition and buffering, recognition systems for gestures, facial expressions, voice commands, command processing, communication, security, web-based monitoring and error recovery. The team will assess system performance by executing tests on each component separately which will allow them to determine whether the system meets its specified operational and performance standards.

Testing approaches like unit testing, integration testing, system testing, performance testing and user acceptance testing can help test ARIS in many different ways. It is very important since the methods provide a great deal of assistance as researchers can analyze problems faced in the system that would improve overall performance when performing operations under ARIS.

Development testing is extremely important since ARIS system is basically a hands free communication system where user inputs in different forms need to be converted into accurate commands and transmitted.

The test plan will help ARIS prototype development because it will identify system weaknesses and confirm core system operations and establish system trustworthiness.

7. References

- TED University, Department of Computer Engineering, ARIS Project Proposal, Fall 2025.
- TED University, Department of Computer Engineering, ARIS Project Specifications Report, Fall 2025.
- TED University, Department of Computer Engineering, ARIS Analysis Report, Fall 2025.
- TED University, Department of Computer Engineering, ARIS High Level Design Report, Fall 2025.
- TED University, Department of Computer Engineering, ARIS Low Level Design Report, Spring 2026.
- IEEE Computer Society, IEEE Standard for Software and System Test Documentation (IEEE Std 829-2008). Referenced as a general guide for organizing the test plan structure.
- ISO/IEC/IEEE, ISO/IEC/IEEE 29119-3: Software and Systems Engineering – Software Testing – Part 3: Test Documentation. Referenced for general test documentation practices.